S-DES Lab 开发手册

1. 项目概览

本项目使用 Python 标准库实现教学版 S-DES，并通过 Tkinter 提供图形界面。系统按“算法核心—界面编排—测试与报告”分层，便于验证算法正确性和后续扩展。

```
S-DES作业/
├── app.py                    Tkinter 图形程序入口
├── sdes.py                   S-DES 算法、文本扩展、破解和碰撞统计
├── demo_results.py           终端测试结果演示
├── tests/test_sdes.py        unittest 自动化测试
└── docs/                     用户指南、开发手册、实验报告和测试结果
```

2. 运行与测试

```
python app.py
python -m unittest discover -s tests -v
python demo_results.py
```

项目不需要第三方依赖，当前实现兼容 Python 3.10+。

3. 算法模块设计

`sdes.py` 将所有二进制数据表示为字符串，例如 `"00101010"`，以保留前导零并与课件中的比特表示一致。

文件顶部定义了题目指定的 `P10`、`P8`、`IP`、`IP_INVERSE`、`EP_BOX`、`SP_BOX`、`SBOX_1` 和 `SBOX_2`。更换 S 盒版本会导致交叉测试结果不一致。

核心公开接口：

函数	输入	输出	说明
`derive_subkeys(key)`	10 位密钥	`(K1, K2)`	生成轮密钥
`round_function(right_half, subkey)`	4 位右半、8 位子密钥	4 位结果	实现 `F(R,K)`
`encrypt_block(plaintext, key)`	8 位明文、10 位密钥	8 位密文	单分组加密
`decrypt_block(ciphertext, key)`	8 位密文、10 位密钥	8 位明文	单分组解密
`encrypt_text(text, key)`	ASCII 文本、密钥	8 位分组列表	逐字节加密
`decrypt_text(blocks, key)`	密文分组、密钥	ASCII 文本	逐字节解密

函数	输入	输出	说明
`brute_force_keys(pairs)`	明密文对	候选密钥列表	枚举 1024 个密钥
`analyze_collisions(plaintext)`	8 位明文	`CollisionAnalysis`	统计密钥碰撞

单分组加密流程为：

```
P → IP → fK(K1) → SW → fK(K2) → IP⁻¹ → C
```

解密复用相同结构，将轮密钥顺序改为 `K2` 后 `K1`。

4. 输入校验与异常

私有函数 `_validate_bits(bits, expected_length, name)` 集中处理固定长度二进制输入。算法层使用 `ValueError` 报告非法长度、非法字符、ASCII 范围错误和空明密文对；界面层捕获异常并使用对话框反馈，因此 `sdes.py` 不依赖 Tkinter。

5. GUI 架构

`SdesApplication` 继承自 `tk.Tk`。界面由顶部深蓝实验室标题、浅色内容区和 ttk Notebook 组成：

```
SdesApplication
├── _build_interface()        顶部标题和标签页
├── _build_basic_tab()        单分组加解密
├── _build_text_tab()         ASCII 扩展
├── _build_crack_tab()        暴力破解
└── _build_collision_tab()    碰撞分析
```

颜色和字体集中配置在 `_configure_styles()`，后续调整主题时优先修改该方法。按钮区分主操作和次操作，顶部状态条展示“算法就绪”，输入提示和结果区保持明确层级。

每个页面的事件函数只负责收集输入、调用 `sdes.py` 和更新控件；密码运算、编码转换和统计逻辑均保留在算法模块中，避免界面与核心耦合。

6. 增加功能的建议

6.1 过程追踪

可新增 `encrypt_block_with_trace()`，返回 `IP(P)`、两轮中间状态和最终密文，不改变现有 `encrypt_block()` 的返回值，避免破坏已有测试。

6.2 文件和网络扩展

文件扩展可按字节读取并复用 `encrypt_block`；网络扩展可将密文分组通过 Socket 传输。Socket、协议编解码和 Tkinter 事件循环应分模块，避免阻塞界面。

6.3 大规模破解

当前 1024 个候选密钥很快完成。若以后扩大密钥空间，应使用线程或 `after()` 分批工作，并通过消息队列将进度交回主线程，不能在工作线程直接更新 Tkinter 控件。

7. 测试策略

`tests/test_sdes.py` 使用标准库 `unittest`，不依赖图形界面。覆盖密钥扩展、已知向量、全部 256 个明文分组的加解密互逆、ASCII 往返、暴力破解和密钥碰撞。

新增算法功能时，至少添加一个正常输入、一个边界输入和一个非法输入测试。修改变换表、轮函数或密钥扩展后必须重新运行全部测试。

8. 代码规范

- 函数和变量使用 `snake_case`，类使用 `PascalCase`。
- 算法常量使用全大写，例如 `P10`、`SBOX_1`。
- 二进制数据保持字符串，避免隐式丢失前导零。
- UI 事件方法负责交互，密码学运算只放在 `sdes.py`。
- 注释说明算法意图和特殊约定，避免重复描述显而易见的代码。